

第1章 Linux 基础

1.1 Linux 操作系统介绍

Linux 是一种自由开源的操作系统内核，同时也常被用来指基于该内核构建的完整操作系统环境。操作系统是计算机系统中最基础的软件层，负责管理硬件资源、调度程序运行并为应用软件提供统一接口。与常见的 Windows 和 macOS 不同，Linux 以开源方式发布，用户可以查看、修改和再发布源代码，这使其在教学、科研与工程领域具有重要地位。

由于其稳定性强、可定制程度高且对多种硬件平台具有良好支持，Linux 被广泛应用于服务器、嵌入式系统、机器人平台与高性能计算环境中。与此同时，Linux 也提供了完善的命令行工具与开发工具链，是学习操作系统原理和开展软件开发实践的重要平台。本节将从 Linux 的发展历程与系统特点入手，建立对 Linux 操作系统的整体认识。

1.1.1 Linux 操作系统的发展

1. Linux 系统的诞生与发展历程

Linux 的起源可以追溯到 1991 年。芬兰赫尔辛基大学的学生 Linus Torvalds 在学习操作系统课程的过程中，出于兴趣编写了一个类 Unix 内核，并于当年 9 月在网络上首次发布。这一项目很快吸引了全球程序员的关注，开发者们通过互联网协作不断扩展和完善该系统。

随着 GNU 项目的自由软件组件（如编译器、工具链和用户空间程序）的加入，Linux 内核逐渐形成了完整的操作系统环境。由于其源代码完全开放，全球开发者能够自由修改与分发，推动了 Linux 的快速迭代与广泛传播。

进入 21 世纪以后，Linux 不仅在传统的服务器和高性能计算机中广泛使用，还逐渐扩展到移动互联网（如 Android 系统）、嵌入式设备、云计算平台和超级计算机集群，形成了庞大的生态体系。

2. Linux 是类 Unix 系统

Unix 操作系统诞生于 20 世纪 70 年代，以简洁、高效和模块化设计闻名，成为现代操作系统的重要源头。Linux 的设计理念深受 Unix 影响，其文件系统层次结构、进程管理方式、权限控制机制以及命令行操作风格都与 Unix 高度相似。

因此，Linux 被称为类 Unix 系统（Unix-like）。这意味着，熟悉 Unix 的用户能够较为容易地使用和迁移到 Linux 环境。同时，Linux 还在开源社区的推动下不断创新，例如引入了更强大的内核模块机制、多样化的文件系统支持和广泛的硬件适配能力，使其在继承 Unix 优点的同时，具备了更强的适应性与生命力。

1.1.2 Linux 发行版概念

1. 发行版概念

Linux 本质上是一个内核，而要成为一个完整可用的操作系统，还需要搭配各种系统工具、软件包管理器、桌面环境以及应用软件。不同的组织和社区基于同一内核，整合了各自的工具链和应用，形成了可安装和使用的 Linux 发行版（Distribution，简称 Distro）。

Linux 发行版的出现，使用户可以根据不同的应用场景和需求选择合适的系统。例如，面向桌面的发行版注重易用性和图形化界面，面向服务器的发行版则更强调稳定性和安全性，而面向嵌入式的发行版则追求轻量化和定制性

2. 常见发行版

目前，Linux 社区涌现出众多发行版，以下是几类具有代表性的：

Debian 系列：以稳定性和自由软件理念著称，代表作有 Debian 本身及其衍生的 Ubuntu。Ubuntu 是目前最受欢迎的桌面发行版之一，也在云计算和服务器领域占有重要地位。

Red Hat 系列：由 Red Hat 公司维护，注重企业级应用和商业支持。其社区版 Fedora 和企业版 Red Hat Enterprise Linux (RHEL) 广泛应用于数据中心。衍生版本 CentOS 也长期被企业和开发者使用。

Arch Linux 系列：以简洁和高度可定制性闻名，面向有经验的用户。其滚动更新模式保证了软件的新性。衍生版本 Manjaro 提供了更加友好的使用体验。

其他特色发行版：如 openSUSE（兼顾桌面与服务器）、Kali Linux（专注于渗透测试与安全研究）、Alpine Linux（轻量级，适合容器环境）等。

通过不同的发行版，Linux 实现了从个人学习、桌面办公到企业部署、云原生应用等多层次的覆盖，为用户提供了多样化的选择。

1.2 Linux 环境配置及使用

1.2.1 在 Windows10/11 配置 WSL2

WSL（Windows Subsystem for Linux，Windows Linux 子系统）是微软在 Windows 系统中提供了一种兼容环境，使用户无需安装双系统或传统虚拟机，就可以直接运行 Linux 发行版及其常用命令行工具。通过 WSL，开发者可以在同一台 Windows 电脑上同时使用 Windows 应用与 Linux 开发工具链，从而简化跨平台开发与学习环境的搭建过程。

WSL 目前主要有两个版本：WSL1 和 WSL2。WSL1 通过将 Linux 系统调用转换为 Windows 系统调用来实现兼容运行，不包含真正的 Linux 内核，因此启动快、资源占用小，但在系统兼容性方面存在一定限制。WSL2 则采用轻量级虚拟化技术，在后台运行一个精简的真实 Linux 内核，从而显著提升了兼容性与功能完整性，能够更好支持现代开发工具与复杂软件栈。

由于在兼容性与性能方面的改进，WSL2 已成为当前主流与推荐的使用方式。

1. 安装 wsl2

1) 虚拟化设置

在 windows 上安装 WSL2 前，首先确保虚拟化选项已经开启。具体的，打开任务管理器，选择“性能”，选中 CPU，然后查看右下角“虚拟化”是否已经启用，如下图 2.2.1-1 所示：

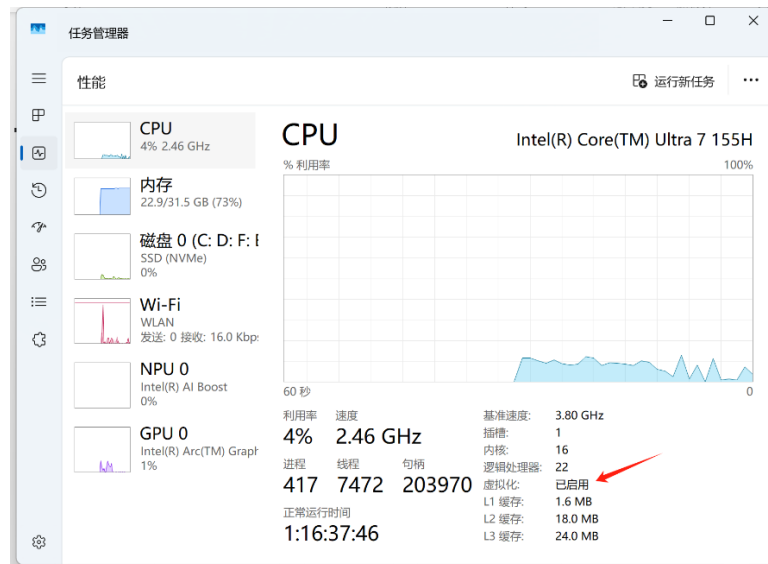


图 2.2.1-1

这个如果没有开启的话，需要进电脑的 BIOS 设置里开启，具体可以搜索对应品牌的电脑如何在 BIOS 中开启虚拟化。

依次打开：控制面板->程序->启用或关闭 windows 功能，开启 Windows 虚拟化和 Linux 子系统（WSL2）以及 Hyper-V，具体按照下图 2.2.1-2 进行勾选：

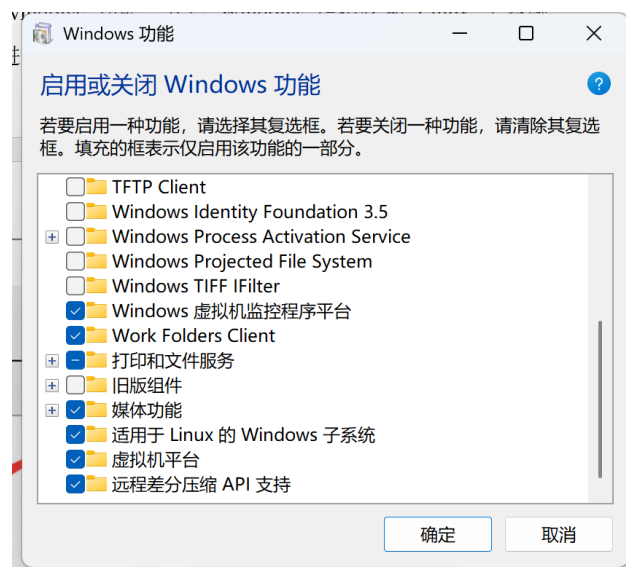


图 2.2.1-2

勾选完成后单击确定按钮，Windows10 会自己下载些东西，并提示重启。等电脑彻底重启完以后，进行后续操作。

2) 切换和更新 wsl2

用管理员权限打开 Windows Power Shell，然后输入下面的命令切换和更新 wsl2：

```
wsl --set-default-version 2
```

```
wsl --update
```

完成后重启系统

2. 安装 ubuntu

1) 下载 Ubuntu22.04

打开 windows 的应用商店，然后搜索 Ubuntu22.04 的稳定版，如下图 2.2.1-3 所示：

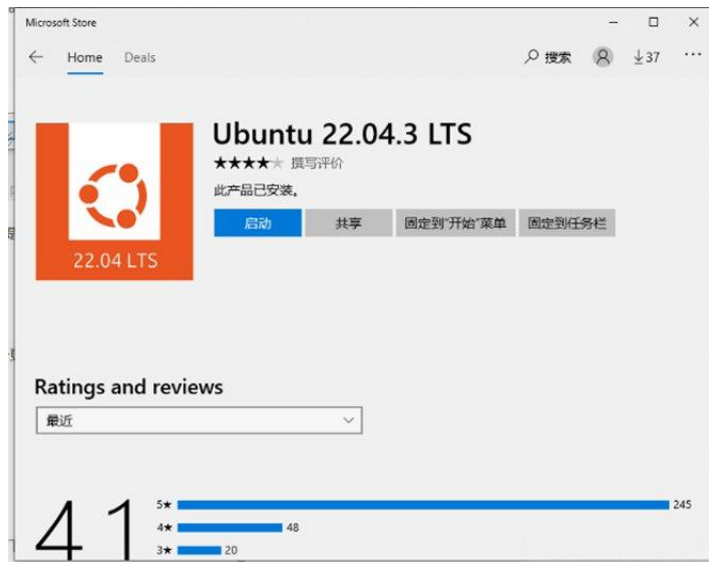


图 2.2.1-3

单机页面上的下载安装即可。

2)配置 Ubuntu22.04

在 Windows 菜单中打开 Ubuntu22.04，如下图 2.2.1-4

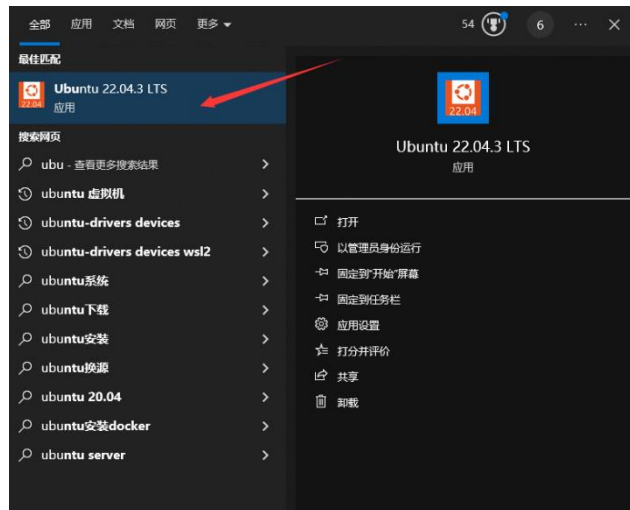


图 2.2.1-4

在打开的 Ubuntu 终端系统中输入下述命令进行更新：

```
sudo apt-get update
```

```
sudo apt-get upgrade
```

如果电脑有英伟达显卡，并且后续想要在 WSL2 的 Ubuntu 中使用该显卡进行 AI 算法研发，那么就需要安装显卡驱动：

```
sudo apt-get install nvidia-cuda-toolkit
```

然后安装显卡工具：

```
sudo apt-get install nvidia-utils-535
```

最后输入 `nvidia-smi` 命令进行测试

1.2.2 安装虚拟机

1.3 Linux 操作系统架构

Linux 操作系统采用 分层架构 设计，核心部分是 内核（Kernel），其上层为 用户空间（User Space）。用户通过 Shell 或应用程序 与内核交互，而内核通过 系统调用（System Call） 向外提供服务。整体上，Linux 架构体现了“内核态与用户态分离”的思想，有助于系统的安全性及稳定性。

Linux 系统架构大体可分为两部分：

内核空间（Kernel Space）：运行操作系统核心代码，直接控制硬件资源。

用户空间（User Space）：运行用户应用程序，通过系统调用请求内核服务。

1.3.1 Linux 系统架构

1. 内核 Kernel

Linux 内核是操作系统的核心组成部分，负责对底层硬件资源进行抽象、管理与调度，是连接硬件与上层应用程序的关键桥梁。所有对 CPU、内存、存储设备以及网络资源的直接控制，均由内核统一完成。

在功能上，内核首先承担着进程管理的职责。它通过调度机制合理分配 CPU 时间片，负责进程的创建、切换与终止，从而实现多任务并发执行。与此同时，内核还实现了完善的内存管理机制，包括虚拟内存、分页管理以及内存的动态分配与回收，使得应用程序能够在有限的物理内存条件下高效运行。

文件系统管理也是内核的重要功能之一。Linux 内核为上层应用提供了统一的文件访问接口，使不同类型的存储设备都可以通过“文件”的形式进行操作，并支持 ext4、xfs、btrfs 等多种常见文件系统。此外，内核通过设备驱动程序对各类硬件进行抽象，将磁盘、网卡、USB 设备等统一表示为可访问的接口，从而屏蔽硬件差异。

在网络功能方面，Linux 内核内部实现了完整的网络协议栈，支持 TCP/IP 等主流网络协议，为各种网络通信应用提供基础支撑。为了使用户程序能够安全地使用上述功能，内核还对外提供了系统调用接口，作为用户空间访问内核服务的唯一入口。

从体系结构上看，Linux 采用的是单内核（Monolithic Kernel）架构，即所有核心功能模块都运行在同一个内核地址空间中。这种设计有利于提升系统性能。同时，Linux 还支持可加载内核模块（Loadable Kernel Module）机制，允许在系统运行过程中动态加载或卸载设备驱动和功能模块，从而在性能与灵活性之间取得平衡。

2. Shell/用户空间

与内核空间相对应，用户空间（User Space）是应用程序运行的区域。普通用户程序无法直接访问硬件资源，而是必须通过系统调用请求内核提供相应的服务。这种限制是操作系统安全机制的重要组成部分。

在用户空间中，Shell 扮演着极其重要的角色。Shell 本质上是一种命令解释器，是用户与操作系统交互的主要接口。用户在终端中输入命令后，Shell 会对命令进行解析，并调用相应的程序或系统服务来完成操作。常见的 Shell 包括 bash、zsh 和 fish 等，它们在基本功能相同的基础上，在交互体验和扩展能力上各有特点。

除了 Shell 之外，用户空间还包含了大量系统和应用组件。例如，图形用户界面系统（如 GNOME、KDE）为用户提供了可视化的操作环境；各类应用程序（如浏览器、文本编辑器和服务器软件）直接运行在用户空间；而系统库（如 glibc）则为应用程序封装了系统调用接口，使开发者无需直接与内核打交道即可使用底层资源。

3. 内核与用户空间的区别

内核空间与用户空间之间存在着严格而清晰的边界，这种分离体现在多个方面。首先是在运行权限上，内核运行于内核态（通常称为 Ring 0），拥有最高权限，可以直接操作硬件资源；而用户程序运行在用户态（Ring 3），其权限受到严格限制，无法直接执行特权指令。

其次，在内存管理方面，内核空间拥有独立且受保护的地址空间，用户程序不能直接访问内核内存。不同用户进程之间也彼此隔离，只能通过内核提供的机制进行调度和通信。这种内存隔离有效防止了进程之间的相互干扰。

在错误影响范围上，两者的差异尤为明显。内核发生错误往往会导致整个系统崩溃，而用户程序的错误通常只会影响自身进程，不会波及系统整体。正是通过这种内核与用户空间的分离设计，Linux 在复杂应用场景下依然能够保持较高的稳定性与安全性。

4. 系统调用

系统调用是连接用户空间与内核空间的关键机制，也是应用程序访问硬件和系统资源的唯一合法途径。用户程序本身并不直接操作硬件，而是通过系统调用向内核发出服务请求。

在实现机制上，用户程序通常先调用标准库函数（如 C 语言中的 `open()`、`read()` 等），这些函数在内部会触发软中断或陷阱指令，使 CPU 从用户态切换到内核态，进入内核中对应的系统调用处理流程。内核完成相应操作后，再将执行结果返回给用户程序。

Linux 提供了丰富的系统调用接口，覆盖了操作系统的各个核心功能领域。例如，在进程管理方面有 `fork()`、`exec()`、`exit()` 和 `wait()` 等调用；在文件操作方面有 `open()`、`read()`、`write()` 和 `close()`；在内存管理方面提供了 `mmap()` 和 `brk()`；设备控制通常通过 `ioctl()` 实现；而网络通信则依赖 `socket()`、`bind()`、`connect()` 等系统调用。

通过系统调用机制，Linux 在保证安全隔离的前提下，为用户程序提供了统一、规范且高效的资源访问方式，使应用开发者无需关注具体硬件细节，也能充分利用操作系统提供的各类能力。

1.3.2 文件系统与目录结构

Linux 操作系统采用统一的文件系统结构来组织和管理数据。无论是系统配置文件、用户数据，还是硬件设备接口，在 Linux 中都以“文件”的形式呈现。这种“万物皆文件”的思想使得操作系统具有简洁、一致的管理方式。理解文件系统与目录结构，是学习和使用 Linux 的基础。

1. 根目录/的组织

Linux 的文件系统以一个根目录（/）为起点，所有文件与目录都在该目录下按层次结构组织。与 Windows 将不同磁盘分配为不同盘符（如 C:、D:）不同，Linux 将所有的存储设备和分区都统一挂载到根目录的树形结构中。

根目录位于最顶层，相当于“树根”，其下的每个子目录都有特定功能和用途。例如，系统启动所需的文件、设备文件、用户文件、临时文件等，都有各自固定的位置。

2. 常见目录及用途

Linux 遵循 Filesystem Hierarchy Standard（FHS，文件系统层次标准），定义了主要目录及其功能。常见目录如下：

<code>/bin</code>	存放基本的二进制可执行文件，如常用命令 <code>ls</code> 、 <code>cp</code> 、 <code>mv</code> 等，供所有用户使用。
<code>/sbin</code>	系统管理员使用的二进制文件，如 <code>ifconfig</code> 、 <code>reboot</code> 等。

/etc	系统配置文件的集中地，例如用户信息文件 <code>/etc/passwd</code> 、网络配置 <code>/etc/network/</code> 等。
/home	普通用户的家目录，每个用户在该目录下拥有一个子目录，用于存放个人数据和配置。
/root	超级用户 <code>root</code> 的家目录。
/lib	系统运行所需的共享库文件和内核模块。
/boot	系统引导相关文件，例如内核映像文件 <code>vmlinuz</code> 和引导加载器配置。
/dev	设备文件目录，硬件设备（磁盘、终端、USB 等）在此以文件形式存在。
/mnt	临时挂载点，常用于挂载外部存储设备。
/media	
/tmp	临时文件目录，系统或应用运行时生成的临时数据会存放于此。
/usr	用户层面的应用和文件，包含大量程序、库和文档，常见子目录有 <code>/usr/bin</code> 、 <code>/usr/lib</code> 。
/var	存放动态变化的数据，例如日志文件 <code>/var/log</code> 、缓存、邮件队列等。

通过这一树状结构，Linux 将系统文件、用户文件与设备接口有机统一。

3. 文件类型概念

Linux 将不同类型的对象都抽象为文件。通过 `ls -l` 命令可以查看文件类型，常见文件类型包括：

普通文件 (-)：文本文件、二进制文件、可执行程序等。

目录文件 (d)：用于组织文件和子目录。

字符设备文件 (c)：以字符流方式访问的设备，如终端、串口。

块设备文件 (b)：以数据块方式访问的设备，如硬盘、U 盘。

符号链接文件 (l)：类似 Windows 的快捷方式，指向另一个文件。

套接字文件 (s)：用于进程间通信 (IPC)。

管道文件 (p)：用于进程间数据传输的特殊文件。

这种统一抽象简化了系统对不同对象的管理，也使得用户能够通过相似的方式操作不同资源。

4. 磁盘及挂载

在 Linux 中，文件系统并不与物理磁盘或分区一一对应，而是通过“挂载”（Mount）机制将不同的存储资源接入到统一的目录树中。物理磁盘通常会被划分为一个或多个分区，每个分区可以格式化为特定类型的文件系统，如 `ext4`、`xfs` 或 `btrfs`，用于负责数据的组织和存储管理。

当用户需要使用某个分区或外部存储设备时，必须将其挂载到已有的目录上。例如，将 U 盘挂载到 `/mnt/usb` 后，用户即可通过访问该目录来读取和写入 U 盘中的内容。为了提高系统的自动化程度，Linux 允许在 `/etc/fstab` 文件中预先定义分区与挂载点，从而在系统启动时自动完成挂载操作。

值得注意的是，Linux 的目录树中不仅包含物理存储设备，还包含若干虚拟文件系统。例如，`/proc` 提供了内核和进程的运行信息，`/sys` 则用于表示内核对象和硬件信息。这些目录并不对应真实的磁盘空间，而是内核向用户空间暴露的信息接口。

通过挂载机制，Linux 将来自不同来源的存储资源整合到同一棵目录树下，使系统在结构上保持高度的一致性和完整性。

1.3.3 Linux 包管理简介

1.4 Linux 常用操作

Linux 系统的日常操作主要通过 **命令行界面（CLI, Command Line Interface）** 完成。虽然现代 Linux 发行版也提供图形用户界面（GUI），但命令行操作更加灵活、强大，是学习和掌握 Linux 的核心内容。

1.4.1 基本操作环境

1. 终端操作

终端操作

终端（Terminal）是用户与 Linux 系统进行交互的主要操作环境。本实验中涉及的命令操作均需在终端中完成。在具有图形界面的 Linux 系统中，用户可以通过快捷键 **Ctrl + Alt + T** 打开终端窗口；在服务器或无图形界面的环境下，用户通常在登录系统后即直接进入终端界面。

在需要对远程服务器进行操作时，常通过 SSH（Secure Shell）方式连接到目标主机，从而获得远程的命令行操作环境。无论是本地终端还是远程终端，其基本使用方式是一致的。

提示符（Prompt）

终端启动后，屏幕上会显示命令提示符，其典型形式如下：

```
username@hostname:~$
```

其中，username 表示当前登录的用户名，hostname 表示主机名，符号 ~ 表示当前所在目录为该用户的家目录。提示符末尾的符号用于区分用户权限：普通用户的提示符通常以 \$ 结尾，而超级用户 root 的提示符则以 # 结尾。在实验过程中，应注意当前使用的用户身份，避免在不必要的情况下以 root 用户执行命令。

2. 基本交互方式

在终端中，用户通过键盘输入命令，并按下 **Enter** 键执行。系统会根据命令的执行结果，在终端中输出相应的信息或错误提示。在命令执行过程中，若需要中断正在运行的程序，可以按下 **Ctrl + C** 组合键；若希望清空当前终端屏幕内容，可使用 **Ctrl + L** 快捷键。

1.4.2 Shell 简介及使用

1. Shell 概念

Shell 是 Linux 系统中的命令解释器，位于用户与操作系统内核之间，是用户与系统交互的重要桥梁。用户在终端中输入的命令，首先由 Shell 进行解析，然后再由系统执行相应的程序或系统调用。

除了提供交互式命令输入功能外，Shell 还是一种功能较为完善的脚本语言。用户可以将多条命令按照一定逻辑组织成脚本文件，用于实现自动化操作，这在实验过程中进行批量处理或重复性任务时尤为重要。

Shell 的核心作用：

命令解释：接收用户输入并执行。

程序控制：提供流程控制语法（条件判断、循环等）。

环境管理：负责环境变量、路径、用户配置的管理

2. 常见 Shell

Linux 支持多种 Shell，不同 Shell 提供的功能和语法略有差别。

常见 Shell 类型：

bash (Bourne Again Shell)	最常用的 Shell，默认在大多数 Linux 发行版中使用。
sh (Bourne Shell)	最早的 Shell，功能较简单。
zsh	扩展性和交互体验更好，支持自动补全和高亮。
csh/tcsh	C Shell 及其改进版，语法类似 C 语言。
fish	现代化 Shell，用户体验友好。

bash 的特点：

Bash 是 Linux 系统中最常用的命令解释器之一，在 Ubuntu、Debian、CentOS 等主流发行版中通常作为默认 Shell 使用。学生在终端中执行的绝大多数命令，都是由 Bash 进行解析和执行的，因此熟悉 Bash 的基本特性和常见操作是进行 Linux 实验的基础。

Bash 支持命令历史记录功能，用户在终端中输入过的命令会被自动保存，可通过方向键在历史命令之间切换，从而避免重复输入。同时，Bash 提供基于 Tab 键的自动补全机制，可对命令名和文件名进行补全，有助于减少输入错误并提高操作效率。

此外，Bash 支持通过别名（alias）机制对常用命令进行简化设置，并具备较为完善的脚本编程能力，能够将多条命令组织成脚本文件，用于完成自动化实验任务或批量操作。

bash 常用操作示例：

查看当前 Shell: `echo $SHELL`

查看系统可用的 Shell : `cat /etc/shells`

命令历史与补全:使用上下键切换历史命令;使用 Tab 键进行命令或文件名补全。

别名定义:`alias ll='ls -l'`

编写简单脚本-创建文件并使用 .sh 输出 Hello, Linux!

```
nano hello.sh
```

```
#!/bin/bash
```

```
echo "Hello, Linux!"
```

保存并退出；

然后赋予执行权限并运行：

```
chmod +x hello.sh
```

```
./hello.sh
```

1.4.3 文件操作

1. 目录操作

2. 文件操作（vim 放在这里讲）

1.4.4 进程管理

1.4.5 包管理

1.5 Linux 用户和用户组

在 Linux 系统中，多用户、多任务是其最显著的特性之一。系统通过用户与用户组的划分，实现对资源的精细化管理与访问控制。掌握用户和权限管理，是保障系统安全与稳定的关键。

1.5.1 用户概述

Linux 系统中的用户（User），指能够登录系统并使用资源的身份。用户信息由系统集中管理，存储在特定配置文件中：

/etc/passwd：记录用户的基本信息（用户名、UID、家目录、默认 Shell 等）。

/etc/shadow：记录用户的密码及密码策略（加密存储）。

Linux 用户可分为以下几类：

超级用户（root）	拥有系统的最高权限，可以执行所有操作，包括管理用户、修改系统配置、安装软件等。其用户名为 root，用户 ID（UID）固定为 0。
普通用户	由系统管理员创建，用于日常操作。权限受限，不能直接修改系统配置或访问其他用户的私有数据。
系统用户	用于运行某些服务或守护进程（如 www-data、mysql），一般不能直接登录系统。这类用户有助于隔离服务，增强系统安全性。

1.5.2 用户组（Group）

在 Linux 系统中，用户组（Group）用于将多个用户组织在一起，从而实现对系统资源的统一管理与权限控制。通过用户组机制，可以方便地为一组用户分配相同的访问权限，提高系统管理的效率与安全性。

系统中的用户组信息统一存储在配置文件 /etc/group 中，该文件记录了系统中所有用户组的名称、组 ID 以及成员列表。

在用户与用户组的关系中，具有以下基本特性：每个用户在创建时都会自动隶属于一个用户组，该组称为主组（Primary Group）；一个用户除了主组外，还可以同时加入多个附加组（Supplementary Groups），从而获得更多资源访问权限。

用户组的主要应用场景体现在文件与目录的权限管理中。Linux 对文件系统对象的访问控制基于三类用户身份：文件的属主（Owner）、文件的属组（Group）、其他用户（Others）。

当多个用户属于同一用户组时，只需为该组设置相应权限，即可实现资源共享，避免对每个用户单独配置权限。

1.5.3 用户管理与权限管理

Linux 系统通过用户管理与权限控制机制，实现对系统资源的精细化访问控制。本节将围绕用户管理与文件权限管理两个方面，介绍常用操作方法。

用户管理主要包括用户的创建、删除、信息修改以及用户信息的查看等操作。相关操作通常需要管理员权限完成。

1. 用户管理

添加用户

使用以下命令可以创建一个新用户，系统会自动为其创建家目录并进行初始配置：

`sudo adduser username`

删除用户

删除用户但保留其家目录：

`sudo deluser username`

删除用户并同时移除其家目录：

`sudo deluser --remove-home username`

修改用户信息

将用户加入指定用户组（作为附加组）：

`sudo usermod -aG groupname username`

修改用户密码：

`sudo passwd username`

查看用户信息

查看指定用户的 UID、GID 及所属用户组：

`id username`

查看当前登录用户：

`whoami`

查看某用户所属的所有用户组：

`groups username`

2. 权限管理

Linux 文件系统通过权限位（Permission Bits）来控制不同用户对文件或目录的访问方式。

用户类别

u (user)： 文件属主

g (group)： 文件属组

o (others)： 其他用户

权限类型

r (read)： 读取文件内容，或列出目录内容

w (write)： 修改文件内容，或在目录中创建、删除文件

x (execute)： 执行文件，或进入目录

一个典型的权限表示形式如下：

`-rwxr-xr--`

其含义为：文件属主具有读、写、执行权限；属组具有读、执行权限；其他用户仅具有读权限。

修改文件权限

使用数字方式修改权限：

`chmod 755 filename`

使用符号方式为属主增加执行权限：

`chmod u+x filename`

修改文件属主与属组

`chown user:group filename`

3. 安全原则

在进行用户与权限管理时，应遵循以下基本安全原则：

日常操作尽量使用普通用户账号，避免直接使用 `root` 用户；

遵循“最小权限原则”，仅为用户或程序分配完成任务所需的最小权限；

系统服务或后台进程应使用专门的系统用户运行，而不应以 `root` 身份执行。

1.6 Linux 网络操作基础

1.6.1 Linux 网络操作概述

Linux 操作系统从设计之初就将网络能力作为核心功能之一，完整实现了 TCP/IP 协议栈，并在服务器、云计算平台、机器人系统与嵌入式设备中得到广泛应用。在实际工程环境中，Linux 不仅承担普通主机的联网功能，还常常作为路由节点、数据采集终端或远程控制平台存在，因此掌握基本的网络操作方法是进行系统部署与调试的重要前提。

在 Linux 中，网络接口被统一抽象为网络设备，由内核统一管理。无论是有线网卡、无线网卡，还是虚拟网卡、容器网络接口，在操作方式上都遵循一致的管理模型。用户通过命令行工具即可完成接口状态查看、IP 地址配置、路由管理、端口检查和链路测试等操作。近年来 Linux 网络工具也在不断演进，传统的 `ifconfig` 与 `netstat` 等工具逐步被 `ip` 与 `ss` 等更现代化的命令替代，这些新工具功能更完整、输出更规范，也更适合自动化脚本与工程环境使用。

本节内容围绕实际使用场景展开，介绍网络状态查看、连通性测试、端口与连接分析、DNS 查询、数据传输与抓包调试等常见操作方法，读者可以直接在终端中实践每条命令以加深理解。

1.6.2 常用网络操作命令

在现代 Linux 系统中，网络管理的核心命令是 `ip` 工具集。它将原来分散在多个命令中的功能统一整合，用于查看与配置接口地址、链路状态以及路由信息。在日常使用中，首先需要学会通过 `ip` 命令查看当前主机的网络配置情况。例如可以使用下面的命令查看所有网络接口与地址信息：

```
ip addr show
ip a
```

如果需要查看系统路由表，可以使用：

```
ip route show
ip r
```

当需要检查网络接口本身的链路状态时，可以执行：

```
ip link show
```

在某些实验或临时调试场景下，也可以直接用 `ip` 命令为网卡临时添加 IP 地址，例如：

```
sudo ip addr add 192.168.1.100/24 dev eth0
```

早期 Linux 教材中常见的 `ifconfig` 命令属于传统工具，目前通常需要额外安装 `net-tools` 软件包才能使用。在维护旧系统或阅读历史文档时仍可能遇到该命令，例如：

```
ifconfig
sudo ifconfig eth0 up
sudo ifconfig eth0 down
```

完成基本配置查看之后，通常需要对网络连通性进行测试。`ping` 命令用于向目标主机发送 ICMP 请求并接收响应，是最基础的连通性检测工具。默认情况下 `ping` 会持续发送数据包，可以通过参数控制次数和间隔：

```
ping baidu.com
ping -c 4 baidu.com
ping -i 2 baidu.com
```

当需要观察数据包经过的路由路径时，可以使用 `traceroute` 命令，它可以逐跳显示到目标主机的转发路径。在部分网络环境中还可以使用 `TCP` 模式或关闭域名解析以提升速度：

```
traceroute baidu.com
traceroute -T baidu.com
traceroute -n baidu.com
```

相比 `traceroute`，`mtr` 提供了动态连续的路径质量统计，更适合课堂实验与故障分析：

```
mtr baidu.com
mtr -r -c 10 baidu.com
```

在排查服务是否正常监听时，需要查看端口与连接状态。现代系统推荐使用 `ss` 命令替代 `netstat`，它速度更快、信息更完整。例如查看所有监听端口和对应进程：

```
ss -tulnp
```

如果只查看 `TCP` 连接或监听端口，可以组合不同参数：

```
ss -t
ss -tuln
```

在需要定位某个端口被哪个进程占用时，可以使用 `lsof` 工具，它从“打开文件”的角度查看网络资源占用关系：

```
lsof -i :80
lsof -p PID
```

`nc`（`netcat`）常被称为网络工具中的“瑞士军刀”，既可以用于端口连通性测试，也可以用于简单的监听与转发实验：

```
nc -zv baidu.com 80
nc -l 8080
```

在涉及域名解析问题时，可以借助 `DNS` 查询工具进行验证。`nslookup` 与 `dig` 都可以完成解析查询，其中 `dig` 输出更详细，更适合教学与排错场景：

```
nslookup baidu.com
dig baidu.com
dig @8.8.8.8 baidu.com
dig -x 8.8.8.8
```

`host` 命令提供了更简洁的查询方式，也常用于快速检查解析结果：

```
host baidu.com
host -t MX baidu.com
```

当需要测试应用层访问能力时，`curl` 与 `wget` 是最常用的下载与请求工具。`curl` 更偏向接口测试与协议调试，可以直接构造 `HTTP` 请求；`wget` 更适合文件下载任务：

```
curl -O http://example.com/file.zip
curl -I baidu.com
```

```
curl -X POST -d "key=value" http://example.com/api
```

```
wget http://example.com/file.zip
```

```
wget -c http://example.com/bigfile.zip
```

```
wget -b http://example.com/bigfile.zip
```

在网络性能测试方面，iperf3 常用于带宽测试，需要一端作为服务端、一端作为客户端协同工作：

```
iperf3 -s
```

```
iperf3 -c 192.168.1.100
```

```
iperf3 -c 192.168.1.100 -t 10 -P 4
```

如果需要对 Web 服务进行压力测试，可以使用 ab 工具进行并发请求实验：

```
ab -n 1000 -c 100 http://example.com/
```

在安全与调试相关场景中，经常会使用端口扫描与抓包工具。nmap 用于扫描主机端口与服务信息，tcpdump 用于底层抓包分析：

```
nmap 192.168.1.100
```

```
nmap -p 1-65535 192.168.1.100
```

```
nmap -sV 192.168.1.100
```

```
tcpdump -i eth0
```

```
tcpdump -i eth0 port 80
```

```
tcpdump -i eth0 -w capture.pcap
```

```
tcpdump -r capture.pcap
```

在主机防火墙管理方面，不同发行版常用不同工具。例如基于 FirewallD 的系统可以使用 firewall-cmd，而 Ubuntu 常用 ufw 进行管理：

```
firewall-cmd --list-all
```

```
firewall-cmd --add-port=8080/tcp --permanent
```

```
firewall-cmd --reload
```

```
ufw status
```

```
ufw allow 8080/tcp
```

```
ufw enable
```

通过这些命令可以覆盖大多数日常网络查看、调试与测试需求。现代 Linux 系统通常推荐优先使用 ip 与 ss 作为基础工具。

1.6.3 远程登录

远程登录是 Linux 网络操作中最常见的应用场景之一。在服务器管理、机器人平台运维和远程实验环境中，用户通常通过远程终端登录目标主机并执行命令。当前主流方式是基于 SSH 协议的安全远程登录机制。

SSH (Secure Shell) 提供加密传输能力，可以在不可信网络中安全地传递账号信息和命令数据。用户通过终端即可直接连接远程主机：

```
ssh user@hostname
```

```
ssh user@192.168.1.10
```

SSH 支持密码认证与密钥认证两种方式。在工程实践中通常推荐使用密钥对认证方式，以减少密码暴露风险并便于自动化脚本调用。SSH 生态中还包含 `scp` 与 `rsync` 等工具，用于在主机之间安全地传输文件：

```
scp file.txt user@host:/path/  
rsync -avz dir/ user@host:/path/
```

Telnet 是早期常见的远程登录协议，但其通信内容以明文形式传输，存在明显安全风险。在现代 Linux 系统中通常不再推荐使用 Telnet，教学与实验环境应优先采用 SSH 方式完成远程操作。

1.6.4 Samba 文件共享和 NFS 文件共享

在局域网或实验集群环境中，多台主机之间往往需要共享文件数据。Linux 常见的共享方案包括 Samba 与 NFS，两者分别面向不同的系统环境与应用场景。

Samba 基于 SMB/CIFS 协议实现，使 Linux 主机能够与 Windows 系统直接进行文件与打印机共享。在跨平台实验室环境中，Samba 常被用于构建公共数据目录。典型配置过程包括安装 Samba 服务程序，编辑配置文件 `/etc/samba/smb.conf` 定义共享目录与访问权限，然后启动服务并设置访问账号。配置完成后，Windows 与 Linux 客户端都可以通过网络路径访问共享目录。

NFS（Network File System）主要面向类 Unix 系统之间的文件共享，具有配置简洁、性能开销小的特点，适合 Linux 服务器或计算节点之间共享数据目录。常见流程是先在服务端安装 `nfs-kernel-server`，随后在 `/etc/exports` 文件中声明可共享的目录与客户端权限，然后启动或重载服务。在客户端主机上，可以通过 `mount` 命令将远程共享目录挂载到本地文件系统，从而像访问本地目录一样访问远程数据：

```
sudo mount 192.168.1.10:/data/share /mnt/share
```

在实际教学与工程实践中，如果环境包含 Windows 主机，通常优先考虑 Samba；如果全部为 Linux 或类 Unix 系统，则 NFS 往往具有更高效率与更低配置复杂度。

1.7 本章基础实践

1.7.1 搭建自己的 Linux 环境

1-4 略，5-7 操作命令见附录

1. 基于虚拟机的 Ubuntu 环境搭建
2. 基于 wsl2 的 Ubuntu 环境搭建
3. 使用 Vim 编写一个 Helloworld 程序
4. 使用 Shell 编写一段程序
5. 操作命令实践
6. 远程登录实践
7. Samba 文件共享和 NFS 文件共享实践

1.8 附录

1.8.1 搭建自己的 Linux 环境

1. 操作命令实践（完整命令示例）

本组命令用于完成一次完整的文件与目录操作练习流程，包括目录创建、文件生成、复制移动、内容查看与权限修改。建议先进入自己的练习目录再执行，避免影响系统文件。

先创建练习目录并进入该目录：

```
mkdir -p ~/linux_practice
```

```
cd ~/linux_practice
```

创建目录与文件：

```
mkdir dirA
```

```
mkdir dirB
```

```
touch file1.txt
```

```
echo "hello linux" > file1.txt
```

查看目录与文件状态：

```
pwd
```

```
ls
```

```
ls -l
```

复制与移动文件：

```
cp file1.txt dirA/
```

```
mv file1.txt file1_main.txt
```

```
mv file1_main.txt dirB/
```

查看文件内容：

```
cat dirB/file1_main.txt
```

```
less dirB/file1_main.txt
```

追加内容并再次查看：

```
echo "second line" >> dirB/file1_main.txt
```

```
cat dirB/file1_main.txt
```

修改文件权限并观察变化：

```
ls -l dirB/file1_main.txt
```

```
chmod 644 dirB/file1_main.txt
```

```
ls -l dirB/file1_main.txt
```

删除文件与目录：

```
rm dirA/file1.txt
```

```
rmdir dirA
```

```
rm -r dirB
```

2. 远程登录实践（SSH 完整操作命令）

本实验假设一台主机作为服务端，一台主机作为客户端。两台机器需要处于同一网络环境中。以下命令在 Ubuntu 环境下测试通过。

在服务端安装并启动 SSH 服务：

```
sudo apt update
```

```
sudo apt install openssh-server -y
```

```
sudo systemctl enable ssh
```

```
sudo systemctl start ssh
```

查看服务状态确认已启动：

```
sudo systemctl status ssh
```

查看服务端 IP 地址：

```
ip a
```

在客户端主机上通过 SSH 登录远程主机：

```
ssh 用户名@服务端 IP
```

首次连接会提示确认指纹，输入 yes 后回车，然后输入密码即可登录。

远程执行命令示例：

```
ssh 用户名@服务端 IP "uname -a"
```

从本机复制文件到远程主机：

```
scp local.txt 用户名@服务端 IP:/home/用户名/
```

从远程主机复制文件到本机：

`scp 用户名@服务端 IP:/home/用户名/remote.txt ./`

使用密钥方式登录（可选进阶实验）：

`ssh-keygen -t ed25519`

`ssh-copy-id 用户名@服务端 IP`

`ssh 用户名@服务端 IP`

3. Samba 文件共享和 NFS 文件共享实践（完整命令示例）

以下示例按“服务端 + 客户端”流程给出，适合两台 Linux 主机互联实验。若与 Windows 主机互通，则只需保留 Samba 部分即可。

Samba 文件共享实验

在服务端安装 Samba：

`sudo apt update`

`sudo apt install samba -y`

创建共享目录并设置权限：

`sudo mkdir -p /srv/samba/share`

`sudo chmod 777 /srv/samba/share`

编辑配置文件：

`sudo vim /etc/samba/smb.conf`

在文件末尾加入：

`[share]`

`path = /srv/samba/share`

`browseable = yes`

`read only = no`

`guest ok = yes`

重启服务：

`sudo systemctl restart smbd`

查看服务状态：

`sudo systemctl status smbd`

客户端访问共享目录（Linux）：

```
sudo apt install cifs-utils -y
```

```
sudo mkdir -p /mnt/samba
```

```
sudo mount -t cifs //服务端 IP/share /mnt/samba -o guest
```

```
ls /mnt/samba
```

NFS 文件共享实验

在服务端安装 NFS 服务：

```
sudo apt update
```

```
sudo apt install nfs-kernel-server -y
```

创建共享目录：

```
sudo mkdir -p /srv/nfs/share
```

```
sudo chmod 777 /srv/nfs/share
```

编辑导出配置：

```
sudo vim /etc/exports
```

加入一行（示例为允许整个网段访问）：

```
/srv/nfs/share 192.168.1.0/24(rw, sync, no_subtree_check)
```

应用配置并启动服务：

```
sudo exportfs -ra
```

```
sudo systemctl restart nfs-kernel-server
```

```
sudo systemctl enable nfs-kernel-server
```

客户端安装并挂载：

```
sudo apt install nfs-common -y
```

```
sudo mkdir -p /mnt/nfs
```

```
sudo mount 服务端 IP:/srv/nfs/share /mnt/nfs
```

```
ls /mnt/nfs
```

查看当前挂载状态：

```
mount | grep nfs
```

取消挂载：

```
sudo umount /mnt/nfs
```